

T02

Seminar In-Class Activity Deck

2026 AI-First · CS423/CSC15003

FIT @ HCMUS · Department of Software Engineering

Web Automation Testing

End-to-end browser tests at speed — and the AI tools that promise to write them for you.

ACTIVITY: Locator Brawl — Hand-crafted vs AI-suggested

Duration: ~25 minutes · For audience teams (3–4 SV)

Why this matters

Web automation is the most common test-automation activity in industry. You will compare two traditional frameworks (Playwright and Cypress) with the AI-augmented direction taken by tools such as Mabl, Testim, and GitHub Copilot. EShop has 22 functional requirements — many touch the browser. Pick three flows to automate and measure not just whether the tests pass, but how maintainable and flake-resistant they are.

Ground Rules for Today's Activity

- Work in your team of 3–4. One designated note-taker per team.
- Read the worksheet first; ask the facilitator before guessing.
- AI tools are allowed — but every output must be cross-checked manually.
- At the end, each team submits ONE minute-paper for audience attendance credit.

What you (the audience) will leave with

Learning outcomes

- Install Playwright and run a parameterised test against the EShop frontend.
- Add a 'self-healing locator' experiment using Testim trial OR a Copilot-suggested locator strategy.
- Measure first-run vs 10th-run flakiness on a network-throttled environment.

Expected takeaways

- ✓ AI locators favour short XPath/CSS but break on small DOM changes.
- ✓ data-test-id attributes still win for long-term maintainability.
- ✓ Self-healing reduces noise but can mask real defects — pair with visual diff.

Tool snapshot — traditional vs AI-augmented

Traditional

Playwright

Microsoft-backed, multi-browser, codegen, trace-viewer; great default.

Selenium 4

Industry standard; W3C WebDriver; widest ecosystem.

Cypress

DX-friendly, time-travel debugger; chrome-family focus.

AI-augmented

Mabl (free tier)

Auto-healing selectors + visual diff on EShop login flow.

Testim AI

Locator stabilisation via attribute-weight model.

GitHub Copilot / Cursor

Generate test scripts from a natural-language scenario — then audit, run, and refactor.

Activity: Locator Brawl — Hand-crafted vs AI-suggested

Audience teams race to write the same Add-to-Cart test two ways and compare results.

What you will need on your laptop

- EShop running locally (frontend + backend) OR ability to connect to the lecturer's stage env.
- Your favourite text editor + a terminal.
- An AI tool of your choice (ChatGPT/Claude/Gemini/Copilot/Cursor — free tier OK).
- The Activity_Worksheet.md provided 3 days before this seminar.

Activity steps — keep tight to the clock

Time	Step
0:00–0:03	Facilitator demos the target flow on EShop (login → add to cart → assert badge).
0:03–0:10	Team A writes the test by hand in Playwright; Team B uses Copilot/Claude.
0:10–0:15	Each team runs the test 3 times on a slow network and records failures.
0:15–0:20	Teams compare locator stability and maintainability.
0:20–0:25	Whole class agrees on a 'good locator' rubric.

Debrief prompts — discuss in 5 minutes

Q1

What surprised you the most?

Q2

Where did the AI tool save you time vs add work?

Q3

Identify ONE failure mode you would prevent next time.

Q4

If you had 1 more hour, what would you investigate next?

Each team writes ONE answer per prompt on the minute paper (handed in at end of class).

Takeaways + further reading

3 things to remember

- ✓ AI locators favour short XPath/CSS but break on small DOM changes.
- ✓ data-test-id attributes still win for long-term maintainability.
- ✓ Self-healing reduces noise but can mask real defects — pair with visual diff.

Further reading

- Playwright docs — Best Practices.
- Bach, J. — 'Test Automation Snake-Oil' (1999) — re-read after AI hype.
- Mabl blog — 'Self-healing tests: what they really do' (technical post).
- Selenium 4 release notes — relative locators.

Submit minute-paper now · Thank you!